

Desenvolvimento para Dispositivos Móveis

Prof. Dr. Rafael dos Passos
Canteri

Módulo 2 - Construção da Interface Gráfica de Aplicativos

Unidade 1 - Principais Layouts, Componentes, Atributos e Unidades



UFMS

AGEAD

Agência de Educação
Digital e a Distância

Gerenciador de layout

- Um gerenciador de *layout* no Android serve para definir a organização visual dos elementos de interface do usuário na tela.
- Ele determina como os componentes visuais, chamados Views, como botões, textos e imagens, serão posicionados e exibidos no dispositivo móvel.

Gerenciador de layout

- Os gerenciadores de *layout* são essenciais para:
 - Organizar os elementos na tela, seja em linhas, colunas ou sobreposições.
 - Ajustar o tamanho e a posição dos elementos para diferentes tamanhos de tela e orientações.
 - Gerenciar o espaço entre os elementos, conhecido como "padding" e "margin".
 - Definir como os elementos reagirão a mudanças no tamanho da tela, garantindo que a interface seja responsiva e adaptável.

Dimensionamento: largura e altura

- Propriedades básicas para qualquer *layout* ou componente:
 - `android:layout_width`
 - `android:layout_height`
- Podem ser:
 - `match_parent` – se adapta ao ancestral
 - `wrap_content` – se adapta ao conteúdo
 - valores numéricos: `dp` (componentes), `sp` (texto).

Margin e Padding

- Atributos mais utilizados no posicionamento de componentes:
 - android:layout_margin - margem externa
 - android:padding - margem interna
- Ambos suportam os sufixos:
 - Top
 - Bottom
 - Left
 - Right

Principais tipos - gerenciador de layout

- Relative Layout
- Constraint Layout
- Linear Layout
- Frame Layout
- Grid Layout
- Table Layout



Fonte: Adapty, 2024

Layouts: Relative Layout

- Usado para *layouts* flexíveis e complexos.
- A posição de cada componente está relacionada com outro componente.
- Atributos importantes:
 - `android:layout_above`
 - `android:layout_toRightOf`
 - `android:layout_alignParentTop`
 - `android:layout_centerInParent`

Layouts: Constraint Layout

- Usado para *layouts* flexíveis e complexos.
- Necessita definição de restrições/regras de posicionamento entre os componentes e a tela.
- Mais flexível que *Relative Layout* e mais fácil de usar com o editor de *layout* do *Android Studio*.
 - Uma evolução do *Relative Layout*.

Layouts: Linear Layout

- Dispõe os componentes na vertical ou na horizontal.
- Requer aninhamento de outros *layouts* para determinadas funções.
- Atributos importantes:
 - `android:layout_weight` (atribui peso ao componente) - ocupa espaço restante
 - `android:orientation` = vertical | horizontal
 - `android:layout_gravity` (externo) - componente
 - `android:gravity` (interno) - conteúdo

Layouts: Frame Layout

- Simples de utilizar.
- Usado quando se deseja colocar apenas um único elemento dentro dele.
- Funciona como camadas.
- Suporta sobreposição de componentes.
- Atributos importantes:
 - `android:layout_gravity` (externo)
 - `android:gravity` (interno)

Layouts: Grid Layout

- Dispõe os componentes em grade.
- Atributos importantes:
 - android:columnCount
 - android:rowCount
 - android:layout_row
 - android:layout_column
 - android:layout_rowSpan
 - android:column_span
 - android:layout_gravity

Layouts: Table Layout

- Mais fácil de controlar que o *Grid Layout*.
- Atributos importantes:
 - TableRow - linha da tabela.
 - android:stretchColumns - quais colunas irão expandir seu conteúdo.
 - android:layout_span - quantas colunas o componente ocupará.

Referências



ADAPTY. **Android Layouts and Views**. 2024. Disponível em: <https://link.ufms.br/ulduh>. Acesso em: 03 mai 2024.

GOOGLE. **Android developers documentation**. Versão 12. 2024. Disponível em: <https://link.ufms.br/quv37>. Acesso em: 02 mai. 2024.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

